

Конвертация регистра

Методы в этой группе выполняют преобразование регистра для строк.

Метод `capitalize()`

Метод `capitalize()` возвращает копию строки `s`, в которой первый символ имеет верхний регистр, а все остальные символы имеют нижний регистр.

Результатом выполнения следующего кода:

```
s = 'foO BaR BAZ quX'
print(s.capitalize())
```

будет:

```
Foo bar baz qux
```

Символы, не являющиеся буквами алфавита, остаются неизменными. Результатом выполнения следующего кода:

```
s = 'foo123#BAR#.'
print(s.capitalize())
```

будет:

```
Foo123#bar#.
```

Метод `swapcase()`

Метод `swapcase()` возвращает копию строки `s`, в которой все символы, имеющие верхний регистр, преобразуются в символы нижнего регистра и наоборот.

Результатом выполнения следующего кода:

```
s = 'FOO Bar 123 baz qUX'
print(s.swapcase())
```

будет:

```
foo bAR 123 BAZ Qux
```

Метод `title()`

Метод `title()` возвращает копию строки `s`, в которой первый символ каждого слова переводится в верхний регистр.

Результатом выполнения следующего кода:

```
s = 'the sun also rises'

print(s.title())
```

будет:

```
The Sun Also Rises
```

Этот метод использует довольно простой алгоритм: он не пытается различить важные и неважные слова и не обрабатывает аббревиатуры и апострофы. Результатом выполнения следующего кода:

```
s = "what's happened to ted's IBM stock?"

print(s.title())
```

будет:

```
What'S Happened To Ted'S Ibm Stock?
```

Метод lower()

Метод `lower()` возвращает копию строки `s`, в которой все символы имеют нижний регистр.

Результатом выполнения следующего кода:

```
s = 'FOO Bar 123 baz qUX'

print(s.lower())
```

будет:

```
foo bar 123 baz qux
```

Метод upper()

Метод `upper()` возвращает копию строки `s`, в которой все символы имеют верхний регистр.

Результатом выполнения следующего кода:

```
s = 'FOO Bar 123 baz qUX'

print(s.upper())
```

будет:

```
FOO BAR 123 BAZ QUX
```

Одно очень важное замечание о методах данной категории состоит в том, что они не изменяют исходную строку. Если вы хотите изменить строку `s`, нужно написать код: `s = s.lower()`. На самом деле тут вы создаёте совсем другой объект в памяти компьютера, просто он со старым названием `s`.

Примечание

Англо-русский словарь:

capitalize — писать прописными буквами, закрепить.

swapcase — обменивать регистр. swap — гл. обмениваться, case — случай, регистр, падеж, дело, расследование...

title — заголовок, титул.

lower — нижний.

upper — верхний.

Поиск и замена

Методы поиска и замены строк внутри других строк.

Каждый метод в этой группе поддерживает необязательные аргументы `<start>` и `<end>`. Как и в строковых срезах действие метода ограничено частью исходной строки, начинающейся с позиции символа `<start>` и продолжающейся вплоть до позиции символа `<end>`, но не включающей ее. Если параметр `<start>` указан, а параметр `<end>` нет, то метод применяется к части исходной строки от `<start>` до конца строки. Если параметры не заданы, то подразумевается, что `<start> = 0`, `<end> = len(s)`.

Метод count()

Метод `count(<sub>, <start>, <end>)` считает количество **непересекающихся** вхождений подстроки `<sub>` в исходную строку `s`.

Результатом выполнения следующего кода:

```
s = 'foo goo moo'

print(s.count('oo'))

print(s.count('oo', 0, 8)) # подсчет с 0 по 7 символ
```

будет:

3

2

Метод startswith()

Метод `startswith(<suffix>, <start>, <end>)` определяет **начинается** ли исходная строка `s` подстрокой `<suffix>`. Если исходная строка начинается с подстроки `<suffix>`, метод возвращает значение `True`, а если нет, то `False`.

Результатом выполнения следующего кода:

```
s = 'foobar'

print(s.startswith('foo'))

print(s.startswith('baz'))
```

будет:

True

False

Метод endswith()

Метод `endswith(<suffix>, <start>, <end>)` определяет **оканчивается ли** исходная строка `s` подстрокой `<suffix>`. Метод возвращает значение `True` если исходная строка оканчивается на подстроку `<suffix>` и `False` в противном случае.

Результатом выполнения следующего кода:

```
s = 'foobar'

print(s.endswith('bar'))

print(s.endswith('baz'))
```

будет:

True

False

Методы find(), rfind()

Метод `find(<sub>, <start>, <end>)` находит **индекс первого вхождения** подстроки `<sub>` в исходной строке `s`. Если строка `s` не содержит подстроки `<sub>`, то метод возвращает значение `-1`. Мы можем использовать данный метод наравне с оператором `in` для проверки: содержит ли заданная строка некоторую подстроку или нет.

Результатом выполнения следующего кода:

```
s = 'foo bar foo baz foo qux'

print(s.find('foo'))

print(s.find('bar'))

print(s.find('qu'))

print(s.find('python'))
```

будет:

0

4

20

-1

Метод `rfind(<sub>, <start>, <end>)` идентичен методу `find(<sub>, <start>, <end>)`, за тем исключением, что он ищет первое вхождение подстроки `<sub>` начиная с конца строки `s`.

Методы `index()`, `rindex()`

Метод `index(<sub>, <start>, <end>)` идентичен методу `find(<sub>, <start>, <end>)`, за тем исключением, что он **вызывает ошибку** `ValueError: substring not found` во время выполнения программы, если подстрока `<sub>` не найдена.

Метод `rindex(<sub>, <start>, <end>)` идентичен методу `index(<sub>, <start>, <end>)`, за тем исключением, что он ищет первое вхождение подстроки `<sub>` начиная с конца строки `s`.

Методы `find()` и `rfind()` являются более безопасными чем `index()` и `rindex()`, так как не приводят к возникновению ошибки во время выполнения программы.

Метод `strip()`

Метод `strip()` возвращает копию строки `s` у которой удалены все пробелы стоящие **в начале и конце** строки.

Результатом выполнения следующего кода:

```
s = '    foo bar foo baz foo qux    '
print(s.strip())
```

будет:

```
foo bar foo baz foo qux
```

Метод `lstrip()`

Метод `lstrip()` возвращает копию строки `s` у которой удалены все пробелы стоящие **в начале** строки.

Результатом выполнения следующего кода:

```
s = '    foo bar foo baz foo qux'

print(s.lstrip())
```

будет:

```
foo bar foo baz foo qux_ _ _ _ _
```

Метод rstrip()

Метод `rstrip()` возвращает копию строки `s` у которой удалены все пробелы стоящие **в конце** строки.

Результатом выполнения следующего кода:

```
s = '    foo bar foo baz foo qux'

print(s.rstrip())
```

будет:

```
_ _ _ _ _foo bar foo baz foo qux
```

Методы `strip()`, `lstrip()`, `rstrip()` могут принимать на вход опциональный аргумент `<chars>`. Необязательный аргумент `<chars>` – это строка, которая определяет набор символов для удаления.

Метод replace()

Метод `replace(<old>, <new>)` возвращает копию `s` **со всеми** вхождениями подстроки `<old>`, замененными на `<new>`.

Результатом выполнения следующего кода:

```
s = 'foo bar foo baz foo qux'

print(s.replace('foo', 'grault'))
```

будет:

```
grault bar grault baz grault qux
```

Метод `replace()` может принимать опциональный третий аргумент `<count>`, который определяет количество замен.

Результатом выполнения следующего кода:

```
s = 'foo bar foo baz foo qux'

print(s.replace('foo', 'grault', 2))
```

будет:

```
grault bar grault baz foo qux
```

Классификация символов

Методы в этой группе классифицируют строку на основе содержащихся в ней символов.

Метод `isalnum()`

Метод `isalnum()` определяет, состоит ли исходная строка из буквенно-цифровых символов. Метод возвращает значение `True` если исходная строка является непустой и состоит **только** из буквенно-цифровых символов и `False` в противном случае.

Результатом выполнения следующего кода:

```
s1 = 'abc123'

s2 = 'abc$*123'

s3 = ''

print(s1.isalnum())

print(s2.isalnum())

print(s3.isalnum())
```

будет:

```
True

False

False
```

Метод `isalpha()`

Метод `isalpha()` определяет, состоит ли исходная строка из буквенных символов. Метод возвращает значение `True` если исходная строка является непустой и состоит **только** из буквенных символов и `False` в противном случае.

Результатом выполнения следующего кода:

```
s1 = 'ABCaBc'
```

```
s2 = 'abc123'
```

```
s3 = ''
```

```
print(s1.isalpha())
```

```
print(s2.isalpha())
```

```
print(s3.isalpha())
```

будет:

```
True
```

```
False
```

```
False
```

Метод isdigit()

Метод `isdigit()` определяет, состоит ли исходная строка **только** из цифровых символов. Метод возвращает значение `True` если исходная строка является непустой и состоит **только** из цифровых символов и `False` в противном случае.

Результатом выполнения следующего кода:

```
s1 = '1234567'
```

```
s2 = 'abc123'
```

```
s3 = ''
```

```
print(s1.isdigit())
```

```
print(s2.isdigit())
```

```
print(s3.isdigit())
```

будет:

```
True
```

```
False
```

```
False
```

Метод islower()

Метод `islower()` определяет, являются ли **все** буквенные символы исходной строки строчными (имеют нижний регистр). Метод возвращает значение `True` если все буквенные символы исходной строки являются строчными и `False` в противном случае. **Все неалфавитные символы игнорируются!**

Результатом выполнения следующего кода:

```
s1 = 'abc'

s2 = 'abc1$d'

s3 = 'Abc1$D'

print(s1.islower())

print(s2.islower())

print(s3.islower())
```

будет:

```
True

True

False
```

Метод isupper()

Метод `isupper()` определяет, являются ли **все** буквенные символы исходной строки заглавными (имеют верхний регистр). Метод возвращает значение `True` если все буквенные символы исходной строки являются заглавными и `False` в противном случае. **Все неалфавитные символы игнорируются!**

Результатом выполнения следующего кода:

```
s1 = 'ABC'

s2 = 'ABC1$D'

s3 = 'Abc1$D'

print(s1.isupper())
```

```
print(s2.isupper())
```

```
print(s3.isupper())
```

будет:

```
True
```

```
True
```

```
False
```

Метод isspace()

Метод `isspace()` определяет, состоит ли исходная строка **только** из пробельных символов. Метод возвращает значение `True` если строка состоит только из пробельных символов и `False` в противном случае.

Результатом выполнения следующего кода:

```
s1 = '      '
```

```
s2 = 'abc1$d'
```

```
print(s1.isspace())
```

```
print(s2.isspace())
```

будет:

```
True
```

```
False
```